

A Clean, ComfyUI-Native Flux.2 ControlNet Integration with Explicit Non-Recursive Multi-Control Composition

Research Background, Architecture, Implementation Methodology, Validation, and Technical
Freeze Record

José Luis Córdova

6 July 2026

Abstract

This paper documents the research and implementation of a clean COMFYUI-native integration for the Alibaba-PAI `FLUX.2-dev-Fun-Controlnet-Union` checkpoint, followed by an explicit non-recursive method for applying more than one control condition through a shared FLUX.2 ControlNet side model. The work began from three complementary sources: the pinned COMFYUI FLUX.2 implementation and model-management lifecycle, the VideoX-Fun reference implementation that defines the intended control-side mathematics, and an experimental ComfyUI adaptation that demonstrated the practical feasibility of intercepting FLUX.2 execution. The resulting implementation retains the useful mathematical structure while rejecting global monkey-patching, replacement of `Flux.forward_orig`, unmanaged device movement, persistent mutable forward state, and recursive child-ControlNet chaining.

Checkpoint inspection established that the released ControlNet file is a compact side branch rather than a duplicated base transformer: 76 BF16 tensors, 4,116,248,576 parameters, approximately 7.667 GiB of tensor data, a 260-to-6144 control-input projection, and four control transformer blocks corresponding to native double blocks 0, 2, 4, and 6. The implementation therefore models only this compact branch, owns it through `CoreModelPatcher`, transports requests with a stateless `TransformerOptionsHook`, evaluates the branch inside a per-forward closure, and injects the resulting residuals after the corresponding native blocks.

Two major milestones were validated. First, a complete single-ControlNet path was demonstrated, including VAE encoding, 260-channel control-context construction, side-branch execution, timestep-range control, exact zero-strength bypass, and coherent residual influence on generated images. Second, a two-branch Orchestrator was demonstrated using one shared checkpoint and one shared model patcher, independent images and schedules, sequential flat child evaluation, weighted residual composition, and a single native injection per block. A stringent equivalence test showed that two identical branches at strengths 0.5 and 0.5 produced an image pixel-for-pixel identical to one branch at strength 1.0: zero differing pixels, zero maximum channel difference, and zero mean absolute difference at 512 by 768 pixels.

The final frozen design also uses a thread-safe lazy checkpoint handle. Text encoding completes before the 7.7 GiB side model is materialized during sampling-model discovery, resolving model-lifecycle pressure while preserving one shared owner across shallow copies and Orchestrator branches. The frozen scope is control-image operation on the FLUX.2-dev family, with single-control and two-branch explicit non-recursive composition. Inpainting, reference-latent parity, generalized branch counts, and substantive multi-GPU work remain intentionally outside this milestone.

Contents

1	Purpose and Scope	3
2	Research Context	3
2.1	The integration gap	3
2.2	Why the established Flux.1 solution could not simply be reused	4
3	Reference-Locking and Checkpoint Forensics	4
3.1	Source-locking method	4
3.2	Checkpoint manifest	5
3.3	Recovered side-branch structure	5
4	Design Requirements	6
4.1	Host-native lifecycle	6
4.2	No global forward replacement	6
4.3	Composable block interception	7
4.4	Per-forward state only	7
4.5	Explicit non-recursive multi-control ownership	7
5	Implementation Architecture	7
5.1	Module responsibilities	7
5.2	Overall data flow	8
5.3	Lazy checkpoint ownership	9
5.4	Control-image encoding and 260-channel context	10
5.5	Native tensor reuse	10
5.6	Stateless wrapper delivery	11
5.7	Single-control residual injection	11
5.8	Timestep scheduling	12
5.9	Base-model compatibility guard	12
6	Milestone I: Complete Single-Control Capability	12
6.1	Staged validation methodology	12
6.2	Foundation identity result	13

6.3	Execute-only tensor result	13
6.4	Control influence result	13
6.5	Zero-strength and scheduling results	14
7	Milestone II: Explicit Non-Recursive Multi-Control Composition	14
7.1	Why an Orchestrator rather than chained Apply nodes	14
7.2	Composite owner	14
7.3	Flat evaluation	15
7.4	Weighted residual composition	15
7.5	Pixel-exact equivalence test	16
7.6	Successful Orchestrator diagnostics	16
8	Final Lazy-Loading Lifecycle	17
8.1	Motivation	17
8.2	Validated ordering	17
9	Validation Environment and Results	17
9.1	Environment	17
9.2	Validation matrix	18
9.3	Representative memory behavior	19
10	Safety, Compatibility, and Invariants	19
10.1	Preserved invariants	19
10.2	Deliberately unsupported paths	19
11	Engineering Significance	20
11.1	A sidecar rather than a replacement transformer	20
11.2	Host extension mechanisms are sufficient	20
11.3	Non-recursive composition is algebraically transparent	20
11.4	Lazy ownership is part of correctness	20
12	Technical Freeze Statement	20
13	Recommended Next Research Directions	21

A Regression Checklist for the Frozen Baseline	22
B Compact Mathematical Summary	22

1 Purpose and Scope

The objective of this work was not merely to make a particular checkpoint produce an image. The objective was to integrate a large FLUX.2 ControlNet into COMFYUI in a way that respects the host application’s model lifecycle, extension interfaces, dynamic VRAM behavior, block-replacement composition, and future compatibility.

The word *clean* is used precisely in this paper. It means that the implementation:

- does not globally replace or monkey-patch `Flux.forward_orig`;
- does not copy the full base FLUX.2 transformer into a replacement model;
- does not manually force the side model between CPU and CUDA inside the sampling path;
- does not keep per-forward residual tensors in global or persistent hook state;
- does not overwrite unrelated block replacements already present in `transformer_options`;
- exposes the side model through COMFYUI’s normal control and model-patcher lifecycle; and
- supports multiple control branches through an explicit flat owner rather than a recursive `previous_controlnet` chain.

The term *non-recursive composition* also has a strict meaning. Each child control is detached from any prior-control chain, is evaluated independently from the same native prepared FLUX.2 tensors, and contributes weighted residuals to one explicit composite. The host sees one composite control owner, not a nested chain of controls calling controls.

This paper records the validated state as a technical freeze. It does not claim an exhaustive global priority search. It does establish a dated, reproducible engineering record for a clean COMFYUI-native FLUX.2 ControlNet implementation and for explicit non-recursive multi-control composition.

2 Research Context

2.1 The integration gap

The Alibaba-PAI checkpoint was published for FLUX.2-dev and described as a union ControlNet that is applied to four double blocks and supports multiple control modalities, including common edge, depth, pose, line, scribble, grayscale, and inpainting conditions [2]. At the beginning of this work, the checkpoint was not recognized by the ordinary COMFYUI ControlNet loader as a native supported format. A working path therefore required both model-architecture interpretation and host integration.

Two public implementations were especially informative:

- **VideoX-Fun** supplied the intended FLUX.2 ControlNet mathematics and checkpoint structure. Its combined transformer implementation showed a four-block side branch, a 260-channel control context, a first-block `before_proj`, four `after_proj` residual heads, and injection at double blocks 0, 2, 4, and 6 [3].
- **comfyui-flux2fun-controlnet** demonstrated that the side branch could be adapted to a

ComfyUI workflow and that interception of the native FLUX.2 forward path was practical [4]. Its approach was useful as an experiment and reference, but the present work deliberately replaced global runtime patching with native hook and patcher mechanisms.

The host reference was a pinned COMFYUI source tree at commit `2a610155821d670a2d8047e654e5fce96b790eb5`. Pinning the host was important because `ControlBase`, model patching, hook delivery, block replacement, dynamic model loading, and FLUX.2 internals are moving interfaces. The implementation documented here should therefore be understood relative to that pinned architecture [1].

2.2 Why the established Flux.1 solution could not simply be reused

Prior JLC work had already established a reliable non-recursive ControlNet composition discipline for classic and Flux.1-style controls: shallow-copy child controls, detach `previous_controlnet`, evaluate independently, synchronize between children when appropriate, clone the first owned output, and accumulate later weighted outputs in place. That lifecycle and ownership discipline transferred directly.

The model mathematics did not. The relevant FLUX.2 differences included:

- 128-channel native model input rather than the Flux.1 control assumptions;
- patch size 1 in the native FLUX.2 transformer path;
- four-axis positional identifiers and global modulation;
- a 6144-dimensional hidden stream with 48 attention heads of dimension 128;
- eight double-stream blocks followed by 48 single-stream blocks;
- SiLU/SwiGLU-style feed-forward behavior and bias-free projection choices;
- dedicated reference-latent semantics and timestep-zero indexing; and
- a control checkpoint that emits exactly four residuals rather than a conventional long list of down, middle, and up residual tensors.

The transferable insight was therefore architectural rather than mechanical: preserve native base-model ownership and place the side model at carefully selected seams.

3 Reference-Locking and Checkpoint Forensics

3.1 Source-locking method

Before implementation, the research sources were collected into a single archival bundle with disambiguated names and checksums. This avoided collisions among several files named `model.py` or `controlnet.py` and preserved the exact material used to derive the integration. The working set included:

- pinned COMFYUI `comfy/controlnet.py`;
- pinned COMFYUI `comfy/ldm/flux/model.py` and `layers.py`;

- `model_base.py`, `model_patcher.py`, `model_management.py`, `patcher_extension.py`, samplers, and hook infrastructure;
- VideoX-Fun FLUX.2 base transformer, control transformer, VAE, image processor, nodes, and workflows; and
- the experimental Flux2Fun node and patch implementation.

The implementation does not import either third-party project at runtime and does not bundle checkpoint weights. Their role was architectural reference and attribution.

3.2 Checkpoint manifest

A safe tensor-manifest extraction was performed with `safetensors.safe_open`, reading tensor metadata and shapes without materializing the checkpoint on the GPU. The result was decisive:

Table 1: Observed checkpoint structure.

Property	Observed value
Checkpoint	<code>FLUX.2-dev-Fun-Controlnet-Union.safetensors</code>
Tensor count	76
Dtype	BF16 for all 76 tensors
Parameter count	4,116,248,576
Estimated tensor bytes	8,232,497,152
Estimated size	7.667 GiB
Top-level prefixes	<code>control_img_in</code> ; <code>control_transformer_blocks</code>
Control projection	<code>control_img_in.weight</code> : 6144×260
Control blocks	Four: indices 0 through 3
Native injection mapping	Double blocks 0, 2, 4, and 6
Head dimension	128
Head count	$6144/128 = 48$

Equally important was what the checkpoint did *not* contain: it did not contain a duplicated eight-plus-forty-eight-block base FLUX.2 transformer. The file was therefore a genuine compact side branch, even though “compact” remains relative at 4.116 billion parameters.

This finding determined the entire strategy. The correct integration was not to load a replacement full transformer. It was to instantiate precisely the checkpoint’s two top-level module families and attach them to the native base model at runtime.

3.3 Recovered side-branch structure

The parameter tree and VideoX-Fun source together established the following architecture:

1. Project a 260-channel control token into the 6144-dimensional hidden space with `control_img_in`.

2. At the first side block, compute a conditioned branch state

$$c_0 = W_{\text{before}}c + x,$$

where x is the native prepared and projected image-token stream.

3. Run four joint image/text transformer blocks using the native prepared text stream, global modulation outputs, positional embedding, and attention mask.
4. After every side block, project the current branch state through `after_proj` to produce one residual.
5. Inject those four residuals after native double blocks $\{0, 2, 4, 6\}$.

The side branch updates its own text state from one side block to the next. It does not replace the native base model’s text state. Its externally consumed product is the set of four image-stream residual tensors.

4 Design Requirements

The implementation was governed by the following requirements.

4.1 Host-native lifecycle

The side model must be discoverable through `ControlBase.get_models()` and owned by a `CoreModelPatcher`. This allows COMFYUI and DynamicVRAM to decide how much of the 7.7 GiB BF16 model is staged, offloaded, or dynamically moved. Direct calls such as `model.cuda()` or manual whole-model CPU shuttling were prohibited from the execution design.

4.2 No global forward replacement

The native FLUX.2 implementation remains the source of truth for:

- latent patchification and image-token identifiers;
- text normalization and projection;
- timestep, guidance, and pooled-vector embeddings;
- global image/text modulation;
- positional embeddings and attention masks;
- reference-latent and timestep-zero behavior;
- LoRA and model-patcher effects; and
- future host-level arguments and patches.

The integration may observe and augment prepared tensors, but it must not fork the full forward implementation.

4.3 Composable block interception

The selected double blocks may already have replacements installed by another extension. The JLC replacement must therefore wrap an existing replacement if present, call it, and inject only after it returns. It may not overwrite the replacement dictionary entry without preserving the previous callable.

4.4 Per-forward state only

Control requests may be transported through the current invocation's `transformer_options`. Large residual tensors, execution flags, and combined outputs must remain in local closure state created for that specific diffusion forward. This prevents cross-prompt leakage, stale state, and unsafe persistence inside reusable hook objects.

4.5 Explicit non-recursive multi-control ownership

Multiple controls must not be implemented as a recursive chain. A composite owner must:

- shallow-copy configured children;
- force every child's `previous_controlnet` to `None`;
- evaluate each active child independently;
- synchronize between children on CUDA during the validation implementation;
- clone only the first residual when taking ownership;
- accumulate later child residuals with in-place weighted addition; and
- inject one combined tensor per native block.

5 Implementation Architecture

5.1 Module responsibilities

The frozen package is organized around a small set of responsibilities:

Table 2: Logical module map.

Module	Responsibility
<code>model.py</code>	Exact compact side-model parameter tree; native-compatible attention and modulation adapter; four residual outputs.
<code>loader.py</code>	Checkpoint validation, architecture inference, lazy materialization, meta-device construction, shared patcher ownership.
<code>control.py</code>	Single-control <code>ControlBase</code> ; VAE encoding; 260-channel context; scheduling; zero bypass; request creation; diagnostics.
<code>hooks.py</code>	Stateless diffusion wrapper; compatibility checks; block-replacement composition; side-branch execution; single or composed injection.
<code>composition.py</code>	Explicit flat composite owner for multiple detached child controls; model deduplication; composite diagnostics.
<code>jlc_flux2_controlnet_loader_node.py</code>	User-facing checkpoint selection.
<code>jlc_flux2_controlnet_apply_node.py</code>	User-facing single-control conditioning application.
<code>jlc_flux2_controlnet_orchestrator_node.py</code>	Two independent image/strength/range branches sharing one ControlNet model and VAE.

5.2 Overall data flow

```

Loader node
↓ lightweight LazyFlux2ControlHandle
Apply node or two-branch Orchestrator
↓ ControlBase / composite ControlBase attached to conditioning
ComfyUI text encoding completes
↓ sampling-model discovery calls get_models()
Lazy handle materializes one compact side model and one CoreModelPatcher
↓ diffusion-model wrapper receives active request(s)
Native Flux.2 prepares img, txt, vec, positional embedding, and mask
↓ block 0 replacement evaluates side branch(es) once
Residuals retained in the current forward closure
↓ after native blocks 0, 2, 4, 6
Weighted residual(s) injected into native image-token stream
↓ native Flux.2 continues unchanged

```

Figure 1: Frozen integration data flow.

5.3 Lazy checkpoint ownership

The loader node returns a lightweight control object containing a shared, thread-safe checkpoint handle. It checks only that the checkpoint path exists; it does not read tensor contents. The actual model is built later, when COMFYUI gathers additional models for sampling.

The core behavior is represented by the following excerpt:

Listing 1: Deferred shared model ownership.

```
class LazyFlux2ControlHandle:
    def materialize(self):
        if self.control_model_wrapped is not None:
            return self
        with self._lock:
            if self.control_model_wrapped is not None:
                return self

            state_dict = comfy.utils.load_torch_file(
                str(self.checkpoint_path), safe_load=True
            )
            architecture = _inspect_architecture(state_dict)

            model = JLCFlux2ControlModel(
                hidden_size=architecture.hidden_size,
                control_in_dim=architecture.control_in_dim,
                num_blocks=architecture.num_blocks,
                num_attention_heads=architecture.num_attention_heads,
                attention_head_dim=architecture.attention_head_dim,
                operations=operations,
                dtype=inference_dtype,
                device=torch.device("meta"),
            )
            model.load_state_dict(state_dict, assign=True)

            self.control_model_wrapped = comfy.model_patcher.CoreModelPatcher(
                model,
                load_device=load_device,
                offload_device=offload_device,
            )
        return self
```

Three design details are important:

1. **Double-checked locking.** All shallow control copies and all Orchestrator children share one handle. A re-entrant lock prevents duplicate construction if discovery paths converge.
2. **Meta-device construction.** Module objects are created on the meta device, then populated with `load_state_dict(assign=True)`. This avoids allocating an additional initialized 7.7 GiB parameter set while the checkpoint dictionary is resident in system memory.
3. **Atomic publication.** The model and patcher are assigned to the shared handle only after every construction step succeeds. No child can observe partially initialized ownership.

The single-control object materializes only when nonzero strength requires the side model:

Listing 2: Exact zero-strength bypass during model discovery.

```
def get_models(self):
    models = super().get_models()
    if self.strength != 0.0:
        models.append(self.ensure_materialized())
    return models
```

Thus a zero-strength control does not read the checkpoint, stage the side model, encode the control image, execute the side branch, or inject a residual.

5.4 Control-image encoding and 260-channel context

The Apply node and Orchestrator store the original control image in BCHW form and supply the VAE. During the first active denoising call, the control object resizes the image to the pixel dimensions corresponding to the native FLUX.2 latent, encodes it through the supplied VAE, restores the previously active model set, applies the FLUX.2 latent-format transform, broadcasts the batch if required, and moves the resulting control latent to the active device and dtype.

For control-image mode, the union checkpoint expects 260 channels:

$$260 = 128_{\text{control latent}} + 4_{\text{mask}} + 128_{\text{masked latent}}.$$

The frozen control-only path uses the encoded control latent and zero-fills the two inpainting-specific components:

Listing 3: Control-only 260-channel context.

```
zeros_mask = torch.zeros(
    (batch, 4, height, width),
    device=control_latent.device,
    dtype=control_latent.dtype,
)
zeros_masked_latent = torch.zeros_like(control_latent)
stacked = torch.cat(
    [control_latent, zeros_mask, zeros_masked_latent], dim=1
)
control_context = rearrange(stacked, "b c h w -> b (h w) c")
```

For the validated 512-by-768 workflow, this produced:

$$z_c \in \mathbb{R}^{1 \times 128 \times 48 \times 32}, \quad H_c \in \mathbb{R}^{1 \times 1536 \times 260}.$$

5.5 Native tensor reuse

The integration does not reconstruct native embeddings. It waits until FLUX.2 has prepared:

- `img`: projected image tokens, shape $B \times N_i \times 6144$;
- `txt`: projected text tokens, shape $B \times N_t \times 6144$;

- **vec**: native global image/text modulation outputs;
- **pe**: native four-axis positional embedding representation; and
- **attn_mask**: the current native attention mask, if present.

At block 0, the side branch is evaluated once per active request. The model adapter reads ComfyUI's `ModulationOut` objects through their `shift`, `scale`, and `gate` fields and calls the native Flux attention helper. This preserves the host's selected attention backend and native positional-embedding representation.

5.6 Stateless wrapper delivery

A `TransformerOptionsHook` delivers one wrapper through `WrappersMP.DIFFUSION_MODEL`. The wrapper is active only when the current invocation carries JLC control requests. It copies only the dictionaries it must modify, preserving tensor references and unrelated options.

For each target block, the implementation stores any existing replacement and installs a wrapper around it:

Listing 4: Composition-safe native block replacement.

```
def injection_replacement(args, extra_options):
    if block_index == CONTROL_LAYERS[0]:
        _run_side_branches_once(request_states, args)

    if existing_replacement is not None:
        out = existing_replacement(args, extra_options)
    else:
        out = extra_options["original_block"](args)

    return _apply_block_residuals(block_index, out, request_states)
```

This establishes a precise ordering:

1. run the side branch once from native prepared block-0 inputs;
2. execute the native block or the previously installed replacement;
3. inject the corresponding control residual into the returned native image stream; and
4. allow the native FLUX.2 forward method to continue.

5.7 Single-control residual injection

Let R_ℓ be the side residual corresponding to native double block $\ell \in \{0, 2, 4, 6\}$, s be the configured strength, and I_ℓ^{native} be the image stream returned by the native block. The single-control rule is:

$$I_\ell^+ = I_\ell^{\text{native}} + sR_\ell.$$

The residual is validated for rank, batch, hidden dimension, token count, device, and dtype. The in-place injection affects only the leading image-token span represented by the residual:

Listing 5: Validated single-control injection.

```
residual = _validated_residual(block_index, image, request, residual)
image[:, : residual.shape[1]].add_(residual, alpha=strength)
```

Applying strength at the injection seam has an important diagnostic consequence: the side branch produces identical residual tensors for identical inputs regardless of the final strength widget. Strength changes only the amount added to the native stream.

5.8 Timestep scheduling

The control object uses the native `ControlBase.pre_run` conversion from user percentages to model timesteps. If the current denoising timestep lies outside the configured range, no request is created and the native output is preserved. Each branch in the Orchestrator has an independent start and end percentage.

5.9 Base-model compatibility guard

The checkpoint architecture is tied to the 6144-wide, 48-head FLUX.2-dev family. Before execution, the wrapper compares the active base model’s hidden width and head count with the materialized control model. An incompatible base model receives a direct explanatory error rather than a deep matrix-shape failure.

6 Milestone I: Complete Single-Control Capability

6.1 Staged validation methodology

The single-control implementation was developed through successful, deliberately isolated gates. The order separated host-integration risk from control-mathematics risk:

1. **Lifecycle foundation.** Load the exact side-model parameter tree through `CoreModelPatcher`; attach a real `ControlBase`; install no-op replacements at blocks 0, 2, 4, and 6; verify that the output is identical to a bypassed workflow.
2. **Execute-only side branch.** Encode the control image, build the 260-channel context, run all four side blocks, record shapes and norms, but do not inject residuals. Verify that the image remains identical.
3. **Residual injection.** Add the four residuals after native blocks 0, 2, 4, and 6. Verify coherent structural influence and strength response.
4. **Hardening.** Add exact strength-zero bypass, timestep-range behavior, duplicate-request protection, shape validation, and explicit base-architecture checks.
5. **Lifecycle refinement.** Defer checkpoint materialization until sampling-model discovery, after text encoding.

This sequence ensured that every stage had a falsifiable expected result.

6.2 Foundation identity result

The no-op wrapper reached the native FLUX.2 invocation with latent shape (1, 128, 48, 32) and text-context shape (1, 512, 15360). It intercepted double blocks 0, 2, 4, and 6 while returning native outputs unchanged. Repeating the same workflow with the diagnostic Apply node bypassed produced an identical image.

This established that hook delivery, conditioning transport, block replacement, and model discovery did not perturb the base model.

6.3 Execute-only tensor result

At 512 by 768 pixels, the complete side branch produced:

- control latent: (1, 128, 48, 32);
- control context: (1, 1536, 260);
- native projected image tokens: (1, 1536, 6144);
- native projected text tokens: (1, 512, 6144); and
- four residuals, each (1, 1536, 6144).

Representative BF16 residual norms were approximately 687.2, 766.2, 1227.7, and 10368.5. The nonzero values and exact alignment with the native image-token shape established that the compact side model was functioning at the intended seam.

6.4 Control influence result

With residual injection enabled, the output remained coherent while following the source control image's dominant pose, centered framing, torso orientation, and silhouette. Increasing the strength from 0.75 to 1.0 visibly increased structural control while allowing the base model to determine identity, clothing, colors, and fine appearance.

The successful log path confirmed:

Listing 6: Representative successful single-control diagnostics.

```
Native Flux.2 wrapper reached: latent=(1, 128, 48, 32),
context=(1, 512, 15360), reference_latents=0.

Side-branch execution confirmed:
control_context=(1, 1536, 260),
img_tokens=(1, 1536, 6144),
txt_tokens=(1, 512, 6144),
residuals=[4 x (1, 1536, 6144)].

Residual injection confirmed at double blocks (0, 2, 4, 6)
with strength=1; dtype=torch.bfloat16, device=cuda:0.
```

This was the first complete milestone: a clean, working, ComfyUI-managed FLUX.2 ControlNet path.

6.5 Zero-strength and scheduling results

The hardened path produced an exact strength-zero bypass. The log explicitly confirmed that VAE encoding, side-model staging, branch execution, and injection were skipped, and the result matched the no-ControlNet baseline.

A restricted range of 0.0 to 0.5 executed the side branch during the active portion of the denoising schedule and logged a native-output preservation message when the control moved outside its range. This demonstrated that strength and scheduling were both applied at well-defined lifecycle points.

7 Milestone II: Explicit Non-Recursive Multi-Control Composition

7.1 Why an Orchestrator rather than chained Apply nodes

A recursive implementation could have attached one control to conditioning and then attached another whose `previous_controlnet` pointed to the first. That approach was intentionally rejected. It obscures ownership, causes one child to call another, complicates model discovery and cleanup, and makes weighted tensor ownership less explicit.

The implemented node is appropriately named the **JLC Flux2 ControlNet Orchestrator**. It performs the Apply responsibilities for two branches and therefore does not require separate Apply nodes. Its interface contains:

- one shared FLUX.2 ControlNet model input;
- one shared VAE input;
- two control-image inputs;
- independent strength for each branch; and
- independent start and end percentages for each branch.

One model input is both simpler and semantically correct for the frozen checkpoint landscape: the two conditions use the same union side model and share one lazy handle and one eventual `CoreModelPatcher`.

7.2 Composite owner

The Orchestrator creates two configured shallow child controls from the shared loader object. Each child receives its own image, strength, and range. Both are detached:

Listing 7: Conceptual detached-child construction.

```
child = shared_control.copy().set_cond_hint(
    hint_bchw,
    strength=branch_strength,
    timestep_percent_range=(start_percent, end_percent),
    vae=vae,
)
child.previous_controlnet = None
```


The children are stored in one composite **ControlBase**. The composite exposes deduplicated models and one hook path to COMFYUI. It is the sole owner visible in conditioning. It does not permit an already-controlled conditioning input, preventing accidental recursive mixing.

7.3 Flat evaluation

At block 0, the hook gathers all active requests and evaluates each side branch independently from the same native `img`, `txt`, `vec`, `pe`, and `mask`. No child receives another child’s residuals or calls another child’s `get_control`.

For deterministic ownership and controlled peak memory behavior, the validation implementation evaluates branches sequentially and synchronizes CUDA between children when more than one branch is active.

7.4 Weighted residual composition

For branch i , let $R_\ell^{(i)}$ be the residual at block ℓ and s_i its strength. The composite residual is:

$$C_\ell = \sum_{i=1}^n s_i R_\ell^{(i)}.$$

The native stream is modified once:

$$I_\ell^+ = I_\ell^{\text{native}} + C_\ell.$$

The implementation follows the established JLC ownership discipline:

Listing 8: Explicit non-recursive residual ownership and accumulation.

```
first = participants[0]
combined = first["residual"].clone()
if first["strength"] != 1.0:
    combined.mul_(first["strength"])

for participant in participants[1:]:
    combined.add_(
        participant["residual"],
        alpha=participant["strength"],
    )

image[:, :, combined.shape[1]].add_(combined)
```

This algorithm has four desirable properties:

1. child residuals remain independent and unmodified;
2. ownership is taken only when necessary, by cloning the first residual;
3. later contributions are accumulated in place; and
4. the native image stream receives one composed addition per block.

After each block’s residual has been consumed, the closure releases that residual entry, reducing the lifetime of large tensors.

7.5 Pixel-exact equivalence test

The strongest validation used the same control image in both branches with strengths 0.5 and 0.5. The reference was the same workflow with one control at strength 1.0. All other generation parameters were held constant.

Because the side branch is deterministic for identical prepared inputs, both child residual sets were identical. The expected identity was:

$$0.5R_\ell + 0.5R_\ell = 1.0R_\ell.$$

The rendered images were compared as RGB arrays rather than by visual inspection or file hash. The result was:

Table 3: Pixel-array equivalence result.

Metric	Result
Image dimensions	512×768 RGB
Differing pixels	0
Maximum absolute channel difference	0
Mean absolute channel difference	0.0

The PNG byte streams differed because their embedded workflow and prompt metadata differed. The image pixels were exactly identical. This is direct evidence that the Orchestrator’s flat weighted composition reproduces the validated single-control result without recursive chaining.

7.6 Successful Orchestrator diagnostics

The frozen run reported:

Listing 9: Representative non-recursive composition diagnostics.

```
Explicit non-recursive composition prepared:
2 configured branches, 2 active,
strengths=[0.5, 0.5], ranges=[(0.0, 1.0), (0.0, 1.0)].

Composite Flux.2 wrapper reached:
latent=(1, 128, 48, 32), context=(1, 512, 15360).

Flat side-branch evaluation active for composition slots=[1, 2]
with strengths=[0.5, 0.5]; no previous_controlnet recursion is used.

Non-recursive composed residual injection confirmed at double blocks
(0, 2, 4, 6) from 2 branches with strengths=[0.5, 0.5].
```

This was the second complete milestone: more than one control condition applied through an explicit, shared-model, non-recursive composition architecture.

8 Final Lazy-Loading Lifecycle

8.1 Motivation

The compact side model is still approximately 7.7 GiB in BF16. The FLUX.2 text encoder used in the validation stack is also very large and is prepared for dynamic loading. Eagerly reading the ControlNet checkpoint during ordinary graph execution creates unnecessary overlap before the sampling lifecycle has gathered its model set.

The final loader therefore registers only a handle during node execution. Materialization occurs when the active control's `get_models()` is queried for sampling.

8.2 Validated ordering

The final run produced the intended sequence:

Listing 10: Validated lazy-loading order.

```
Registered lazy checkpoint handle [final-lazy-after-text-encoder-v1].
No checkpoint tensors have been read.

Requested to load Flux2TEModel_
Model Flux2TEModel_ prepared for dynamic VRAM loading.

Deferred materialization now reading checkpoint for sampling
[final-lazy-after-text-encoder-v1].

Lazily materialized compact side model: 4.116B params,
4 blocks, hidden=6144, heads=48, dtype=torch.bfloat16.
Shared model ownership is active.
```

After materialization, COMFYUI requested the `JLCFlux2ControlModel` and the native `Flux2` model together and partially loaded them under `DynamicVRAM`. Both Orchestrator children then shared that one patcher.

9 Validation Environment and Results

9.1 Environment

The frozen milestone was validated under the following environment:

Table 4: Primary validation environment.

Component	Configuration
Operating system	Windows 11
Python	3.10.11
PyTorch	2.9.1 with CUDA 13.0 build
GPU	NVIDIA RTX 4090 Laptop GPU, 16 GiB VRAM
System RAM	Approximately 64 GiB
COMFYUI	Pinned commit 2a610155821d670a2d8047e654e5fce96b790eb5
Memory mode	Normal VRAM with DynamicVRAM behavior; no forced <code>-lowvram</code>
Base diffusion model	<code>flux2_dev_fp8mixed.safetensors</code>
Text encoder	<code>mistral_3_small_flux2_fp8.safetensors</code>
VAE	<code>flux2-vae.safetensors</code>
Control checkpoint	<code>FLUX.2-dev-Fun-Controlnet-Union.safetensors</code>
Test resolution	512×768
Diagnostic sampling	Four Euler steps with Flux.2 scheduler
Control execution dtype	BF16

9.2 Validation matrix

Table 5: Successful validation matrix.

Test	Expected property	Observed result
No-op foundation	Hook and block interception do not perturb native generation	Same-seed image matched bypassed workflow; native blocks returned unchanged.
Execute-only side branch	Side model produces correctly aligned nonzero residuals without affecting output	Four BF16 residuals of shape $(1, 1536, 6144)$; image remained unchanged.
Single control, strength 1	Coherent structural influence	Generated image followed the control pose and framing while retaining base-model appearance generation.
Strength 0	Exact and inexpensive bypass	Side model was not staged; VAE encoding, execution, and injection were skipped; baseline preserved.
Range 0.0–0.5	Independent temporal scheduling	Control executed only in the active portion; later denoising calls preserved native output.
Two branches, $0.5 + 0.5$	Flat weighted composition equals one branch at 1.0 for identical inputs	Zero pixel differences at 512×768 RGB.
Lazy loader	Text encoding precedes ControlNet checkpoint materialization	Logged handle registration, text-encoder preparation, deferred checkpoint read, then sampling-model staging.

Test	Expected property	Observed result
Shared model owner	Two Orchestrator branches do not duplicate the 7.7 GiB side model	One lazy handle and one materialized <code>CoreModelPatcher</code> shared across both shallow children.

9.3 Representative memory behavior

The side model was reported as approximately 7851 MB staged. In representative combined runs, COMFYUI loaded part of the native base and side model into the available VRAM and kept more than 20 GB offloaded, with no low-VRAM patch count. This is the expected benefit of exposing the side model through the normal model manager rather than forcing residency manually.

Four-step sampling after models were prepared took approximately 12–15 seconds for a single branch and approximately 20–22 seconds for two independent branches in the validation workflow. These numbers are diagnostic observations, not generalized benchmarks; startup, text encoding, VAE movement, cache state, quantization, and other system conditions materially affect total prompt time.

10 Safety, Compatibility, and Invariants

10.1 Preserved invariants

The frozen implementation preserves the following invariants:

- The native FLUX.2 forward method remains installed and authoritative.
- The side checkpoint is represented by exactly the modules present in its state dictionary.
- All large model ownership is visible to COMFYUI model management.
- The control image is encoded through the user-connected VAE.
- Each diffusion forward owns its own request state and residual tensors.
- Existing block replacements are called, not discarded.
- Reference-latent requests are not partially controlled; the current implementation preserves native output when reference latents are present.
- The Orchestrator does not use `previous_controlnet` recursion among children.
- Identical shared loader inputs produce shared lazy ownership, not duplicated side models.

10.2 Deliberately unsupported paths

The freeze does not include:

- inpainting or mask-driven use of the 4+128 auxiliary control-context channels;
- reference-latent parity with the official combined implementation;

- arbitrary dynamic branch counts beyond the validated two-branch Orchestrator;
- mixed native and JLC Flux.2 control chains;
- compatibility claims for FLUX.2 Klein families;
- substantive multi-GPU execution or performance optimization; or
- public Registry-level packaging and support guarantees.

The code retains inherited compatibility structures where useful, but the validated development target is a single 16 GiB NVIDIA GPU.

11 Engineering Significance

11.1 A sidecar rather than a replacement transformer

The checkpoint manifest enabled a crucial simplification: the full base model did not need to be merged, cloned, or reconstructed. The integration can be understood as a sidecar that consumes native prepared tensors and returns four residuals. This minimizes divergence from the host and confines checkpoint-specific code to a well-defined module.

11.2 Host extension mechanisms are sufficient

The complete path was implemented with the host’s existing extension mechanisms: `ControlBase` and `CoreModelPatcher` for lifecycle ownership, `TransformerOptionsHook` and `WrappersMP.DIFFUSION_MODEL` for wrapper delivery, and composed DIT block-replacement entries for injection. No core-file edit and no global monkey patch were required.

11.3 Non-recursive composition is algebraically transparent

The pixel-exact $0.5 + 0.5 = 1.0$ result is more than a regression check. It demonstrates that the architecture implements the intended linear residual algebra explicitly and without hidden recursive effects. Each child is a pure contributor to a final weighted sum at the injection seam.

11.4 Lazy ownership is part of correctness

For models of this size, loading order is not merely a performance detail. The lazy handle makes checkpoint materialization occur at the lifecycle stage where COMFYUI is already assembling the sampling model set. It also guarantees that shallow copies and Orchestrator branches converge on one owner. In this design, memory lifecycle and mathematical correctness are treated as one system.

12 Technical Freeze Statement

Frozen milestone — As of 6 July 2026, the following implementation state is accepted as the frozen research milestone:

1. A compact `JLCFlux2ControlModel` exactly matching the released 76-tensor, 4.116-billion-parameter BF16 side checkpoint.
2. A clean COMFYUI-native single-ControlNet path using `ControlBase`, `CoreModelPatcher`, a stateless diffusion wrapper, and composed native block replacements.
3. Correct control-image VAE encoding and construction of the 260-channel control-only context.
4. Side-branch execution from native projected image/text streams, native global modulation, native positional embedding, and native attention backend.
5. Residual injection after native double blocks 0, 2, 4, and 6 with independent strength and timestep range.
6. Exact strength-zero bypass before side-model materialization and VAE encoding.
7. A two-branch JLC Flux2 ControlNet Orchestrator with one shared checkpoint input, one shared lazy model owner, independent images, independent strengths, and independent ranges.
8. Explicit flat child evaluation with no `previous_controlnet` recursion, weighted in-place residual accumulation, and one combined injection per native block.
9. Pixel-exact validation that two identical branches at strengths 0.5 and 0.5 equal one branch at strength 1.0.
10. Deferred checkpoint materialization after text-encoder preparation and during sampling-model discovery.

No further feature work should modify this frozen baseline without first preserving it in version control and retaining a reproducible milestone archive. Subsequent work should branch from this state and treat the single-control identity tests, the control-influence tests, the zero/range tests, the pixel-exact composition test, and the lazy-loading order as mandatory regressions.

13 Recommended Next Research Directions

The freeze intentionally separates proven capability from future expansion. The most logical next directions are:

1. **Generalized Orchestrator branch count.** Extend the validated two-branch flat owner to a bounded dynamic interface while preserving one shared model handle and the same residual algebra.
2. **Inpainting and mask support.** Replace the zero mask and zero masked-latent channels with the official 4-channel mask representation and 128-channel masked-image latent, then validate spatial continuity.
3. **Reference-latent parity.** Reproduce the official behavior in which the side branch operates over the appropriate main/reference token sequence, respecting native reference identifiers, masks, and timestep-zero indexing.
4. **VAE-encode reuse.** Detect identical control-image/VAE/resolution combinations within one Orchestrator and avoid duplicate encodes without introducing unsafe persistent caches.
5. **Broader compatibility testing.** Validate quantized base-model variants and future FLUX.2-dev releases while retaining explicit architecture guards. Compatibility with Klein should remain a separate research question unless a matching checkpoint is published.

6. **Packaging and public preview.** Publish the frozen source, this paper, a minimal workflow, provenance notes, license review, and clear experimental-scope language in a dedicated repository before considering wider distribution.

Acknowledgments

This work builds upon the native architecture and extension interfaces of COMFYUI, the FLUX.2 model family, the Alibaba-PAI/VideoX-Fun ControlNet checkpoint and reference implementation, and the practical experiment provided by comfyui-flux2fun-controlnet. The implementation strategy also follows the established JLC principle of explicit, ownership-safe, non-recursive ControlNet composition.

A Regression Checklist for the Frozen Baseline

A release or refactor derived from the frozen state should pass all of the following:

1. Start COMFYUI and confirm loader, Apply, and Orchestrator nodes register without importing VideoX-Fun or Flux2Fun.
2. Run a no-control baseline at a fixed seed.
3. Insert a strength-zero Apply node and verify exact pixel identity and no side-model materialization.
4. Run one active control and verify four residuals of the expected token and hidden shape.
5. Verify coherent structural influence at strength 1.0.
6. Restrict the range and verify native preservation outside the active interval.
7. Run the Orchestrator with identical images and strengths 0.5/0.5.
8. Compare its RGB array with the single-control strength-1.0 result and require zero differences.
9. Confirm the lazy log order: handle registration, text-encoder preparation, deferred checkpoint read, shared side-model materialization.
10. Confirm no child Orchestrator control has a non-None `previous_controlnet`.
11. Confirm only one unique side-model patcher is returned for branches sharing the loader.
12. Confirm existing unrelated double-block replacements remain callable and ordered before residual injection.

B Compact Mathematical Summary

For control-only operation:

$$H_c = \text{tokens}(\text{concat}_C [z_c, 0_4, 0_{128}]) \in \mathbb{R}^{B \times HW \times 260}.$$

The side model computes:

$$(R_0, R_2, R_4, R_6) = \mathcal{C}(H_c, x, t, v, p, m),$$

where x is the native projected image stream, t the projected text stream, v the native global modulation, p the positional embedding, and m the attention mask.

For one branch:

$$I_\ell^+ = I_\ell^{\text{native}} + sR_\ell.$$

For n explicitly composed branches:

$$I_\ell^+ = I_\ell^{\text{native}} + \sum_{i=1}^n s_i R_\ell^{(i)}, \quad \ell \in \{0, 2, 4, 6\}.$$

No $R_\ell^{(i)}$ is produced by recursively invoking another child control. All are independent evaluations coordinated by one composite owner.

References

- [1] ComfyUI contributors, *ComfyUI source tree, pinned commit 2a610155821d670a2d8047e654e5fce96b790eb5*, GitHub. <https://github.com/comfyanonymous/ComfyUI/tree/2a610155821d670a2d8047e654e5fce96b790eb5>
- [2] Alibaba-PAI, *FLUX.2-dev-Fun-Controlnet-Union*, Hugging Face model repository. <https://huggingface.co/alibaba-pai/FLUX.2-dev-Fun-Controlnet-Union>
- [3] AIGC Apps / Alibaba contributors, *VideoX-Fun*, GitHub. <https://github.com/aigc-apps/VideoX-Fun>
- [4] Bryan McGuire, *comfyui-flux2fun-controlnet*, GitHub. <https://github.com/bryanmcguire/comfyui-flux2fun-controlnet>
- [5] Black Forest Labs, *FLUX.2-dev*, Hugging Face model repository. <https://huggingface.co/black-forest-labs/FLUX.2-dev>